

Projectdocumentatie – AquaSense (Waterview)

Slim zwembadmonitorsysteem



Sven Van Leemput
Integratieproject - Toegepaste Informatica
AquaSense

Solution Design – AquaSense (Waterview)

Dit document beschrijft het ontwerp op systeemniveau: het geheel, de hoofdonderdelen, de belangrijkste technologiekeuzes met motivatie, en globale datastromen.

1. Probleem, doel en doelgroep

Particuliere zwembadbezitters meten waterkwaliteit vaak zelden of handmatig; er is weinig inzicht in het verloop. Doel: automatisch meten (o.a. temperatuur, pH; chloor is in de software ook voorzien), gegevens centraal opslaan en tonen in een dashboard uitgewerkt in de applicatie AquaSense (Waterview).

Doelgroep: gebruikers zonder sterke technische achtergrond; de interface moet begrijpelijk blijven.

2. Architectuur van het geheel

2.1 Wat hoort er bij het geheel?

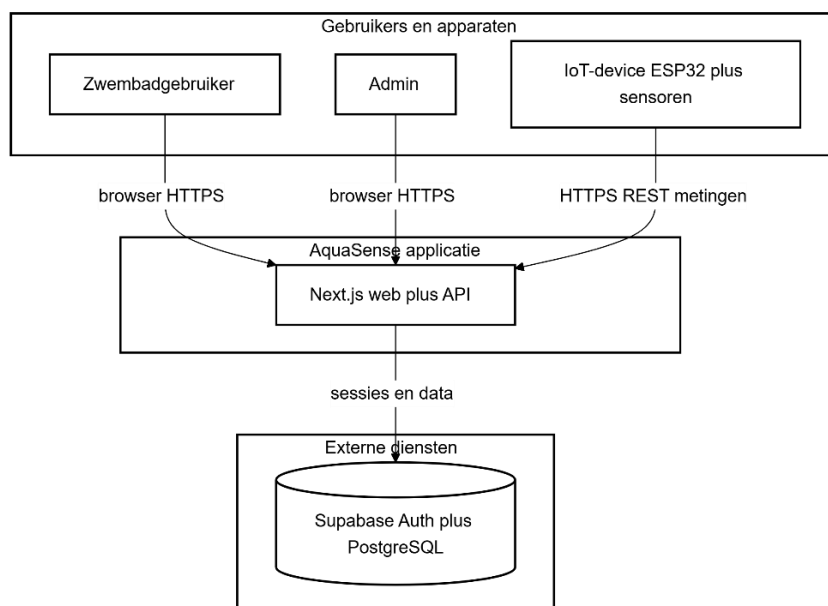
Het systeem heeft vier kernonderdelen, zoals in het voorstel:

Onderdeel	Functie op hoog niveau
IoT-device	Sensoren uitlezen, via WiFi metingen naar de server sturen
Backend	API voor het device en voor beheer; verwerking en regels
Databank	Profielen, toestellen, metingen, drempels, logs, ...
Webdashboard	Inloggen, gebruikers- en adminfunctionaliteit, visualisatie

Implementatie: Backend en web zitten in één Next.js-applicatie (server rendert pagina's én biedt REST-routes voor het device). Dat vermindert het aantal aparte servers en past bij een MVP en een integratieproject.

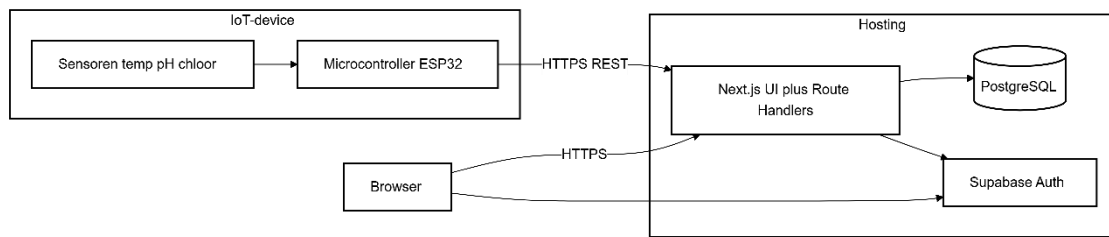
2.2 Systeemcontext

Externe actoren en systemen rond AquaSense:



De gebruiker en de admin werken via de browser. Het IoT-device praat alleen met de applicatie-API. Supabase verzorgt gebruikersauthenticatie en de PostgreSQL-databank.

2.3 Hoofddcomponenten en technologie



2.2 Stack en hosting

Laag	Technologie
Web + API	Next.js 15 (App Router), TypeScript, React
API-stijl	REST, JSON over HTTPS
Gebruikerslogin	Supabase Auth (sessie via cookies, middleware vernieuwt sessie)
Data	PostgreSQL (Supabase), ORM: Prisma
Typische hosting	Webapp: Vercel (zoals in README); auth + DB: Supabase

3. Kerncomponenten

Samenvatting op systeemniveau.

Component	Verantwoordelijkheid	Belangrijk voor het ontwerp
IoT-device	Periodieke metingen: HTTP POST met JSON naar de ingest-route: identiteit via API-sleutel per toestel	Hardware ESP32, DS18B20, pH-module ; firmware staat in repo onder firmware/
Backend	Sleutel en payload controleren; metingen opslaan, admin-operaties, auditlogging op hoofdlijnen	Next.js API + Zod validatie
Database	Relationeel model: profiel, device, measurements, thresholds, system logs, pool/dosis-instellingen	PostgreSQL, geschikt voor veel metingen
Webdashboard	Login: dashboard voor gebruikers; admin-zone voor devices, logs, drempels	Login + dashboard met beheer

4. Architectuurkeuzes en motivatie

4.1 API-model

REST + JSON over HTTPS

- Het device stuurt data via HTTP POST met JSON.
- Dezelfde aanpak wordt gebruikt voor admin- en app-acties. Dit houdt alles consistent.

Geen WebSockets als kern

- Realtime updates zijn geen vereiste voor het MVP.
- De applicatie werkt met opgeslagen data en updates via pagina's en grafieken.
- WebSockets kunnen later toegevoegd worden zonder grote aanpassingen.

Alternatief: MQTT

- MQTT is krachtiger voor grote IoT-systemen, maar voegt extra complexiteit toe.
- Voor dit project is REST eenvoudiger en makkelijker te testen, zeker met een ESP32.

4.2 Databanktype

PostgreSQL (relationeel)

- De data heeft duidelijke relaties: gebruiker → devices → metingen.
- Een relationele database past hier goed bij en maakt queries eenvoudiger.

Prisma als laag

- Prisma zorgt voor typeveilige queries in TypeScript.
- Migraties zijn reproduceerbaar en verminderen fouten.

Alternatief: MongoDB

- Een documentdatabase is flexibel voor tijdsdata, maar maakt relaties en rapportage minder duidelijk in deze context.

4.3 Beveiliging (ontwerp op systeemniveau)

Gebruikers (user/admin)

- Authenticatie gebeurt via Supabase Auth.
- Rollen (USER, ADMIN) worden beheerd in het profiel.
- Admin-routes zijn server-side beveiligd.

IoT-device

- Het device gebruikt een API-sleutel in de header.
- De sleutel wordt in de database opgeslagen als hash (SHA-256), niet als platte tekst.

Transport

- Alle communicatie verloopt via HTTPS.

Validatie

- De backend valideert alle input met Zod.
- Fouten worden teruggestuurd zonder interne details.

Eerste gebruiker = admin

- De eerste gebruiker krijgt adminrechten.
- Voor grotere systemen zijn extra afspraken nodig.

4.4 Motivatie gekoppeld aan het onderzoek

Embedded

- De keuze voor ESP32 en sensoren komt uit onderzoek naar betrouwbaarheid en beschikbaarheid.
- De ESP32 werkt als eenvoudige HTTP-client met WiFi.

System

- PostgreSQL en Supabase zorgen voor stabiele opslag en eenvoudige authenticatie. Dit vermindert de nood aan eigen infrastructuur.

Developer

- Next.js, TypeScript, Prisma en Zod zorgen voor één consistente codebase.
- Dit maakt ontwikkeling sneller en minder foutgevoelig.

5. Datastromen en interacties

5.1 Drie hoofdstromen

Hier zijn de drie belangrijkste flows in het systeem.

Ingest (device → backend → database)

- 1 Het device maakt een JSON-object met metingen
- 2 stuurt dit via HTTP POST naar de ingest-URL.
- 3 Het voegt een token toe voor identificatie.
- 4 De server controleert eerst de sleutel en de inhoud van de data.
- 5 Als alles klopt, slaat hij een nieuwe meting op en werkt hij de *last seen* van het device bij.
- 6 De server geeft een response terug:
 - **201** bij succes
 - **401** bij een foute sleutel
 - **400** bij ongeldige data
 - **500** bij een serverfout

Dashboardgebruiker

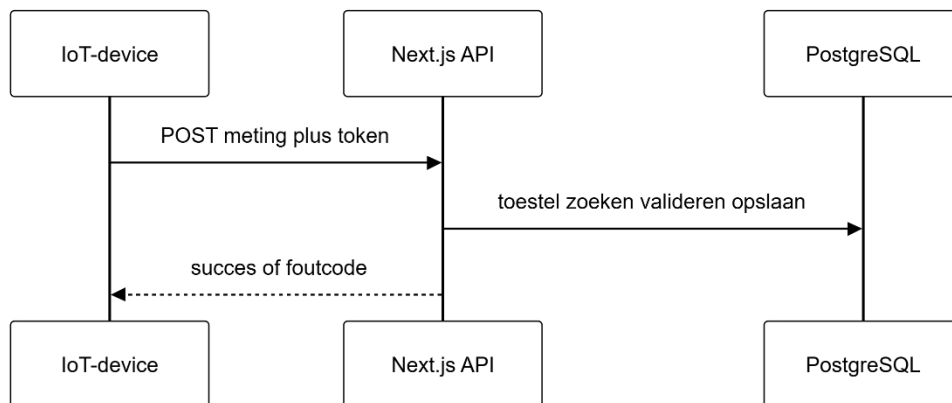
- 1 De gebruiker logt in via Supabase.
- 2 Na login laadt de applicatie het profiel en de bijhorende data uit PostgreSQL.
- 3 De gebruiker ziet een dashboard met status, grafieken en historiek.
- 4 Filters en paginering zorgen ervoor dat grote datasets overzichtelijk blijven.

Admin

De admin volgt dezelfde loginflow als een gewone gebruiker, maar heeft extra rechten.

- 1 nieuwe devices aanmaken (met een API-sleutel die één keer getoond wordt)
- 2 drempels en poolinstellingen aanpassen
- 3 logs bekijken

5.2 Sequentie: meting



5.3 Sequentie: gebruiker bekijkt dashboard

